

การวิเคราะห์ ออกแบบ และจำลองการทำงานของ Microprocessor EC-1

(Lab 2 : The EC-1 Microprocessor)

ด้วยโปรแกรม Logisim-Evolution

การศึกษาโครงสร้างสถาปัตยกรรมของส่วนประมวลผลข้อมูล (Datapath) และหน่วยควบคุม (Control Unit) แบบ Hardwired FSM ที่พัฒนาด้วยวงจร Logic เชิงผสมร่วมกับ D Flip-Flop

จัดทำโดย

- | | | |
|---|---------------------------|------------|
| 1 | นายดรณ์ภพ พิทักษ์กิจไพศาล | 6710301007 |
| 2 | นายอภิสิทธิ์ คงภักดี | 6710301009 |
| 3 | นายธนัท จงธีรธนโชติ | 6710301032 |

เสนอ อาจารย์ศุภาเทพ สวัสดิ์พิศาล

รายวิชา Embedded System Design ภาคการศึกษาที่ 1 ปีการศึกษา 2569

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อนำเสนอผลการศึกษาและปฏิบัติงานในการทดลองที่ 2 หัวข้อ **Microprocessor EC-1 (The EC-1 Microprocessor)** ในรายวิชา Embedded System Design (010153521) โดยมีวัตถุประสงค์หลักเพื่อให้นักศึกษาเข้าใจสถาปัตยกรรมคอมพิวเตอร์อย่างลึกซึ้ง ผ่านการสร้าง Hardware และควบคุมทิศทางข้อมูลของ Microprocessor จำลองขนาดเล็ก

Microprocessor EC-1 เป็นสถาปัตยกรรมขนาด 8 Bit ที่มีโครงสร้างเรียบง่ายแต่ครบถ้วนตามหลักการของ Von Neumann Architecture ซึ่งประกอบด้วยหน่วยควบคุม (Control Unit) แบบใช้ LogicHardware (Hardwired FSM) และส่วนประมวลผลข้อมูล (Datapath) ที่เชื่อมต่อกันอย่างเป็นระบบ การสร้างระบบดังกล่าวบน Software Logisim-Evolution จะช่วยให้นักศึกษาสามารถเชื่อมโยงความรู้เชิงทฤษฎีในเรื่องของสัญญาณนาฬิกา (Clock Cycle), Register (Register), และหน่วยความจำ (ROM) เข้ากับการทำโปรแกรมระดับต่ำ (Assembly Language) ได้อย่างเป็นรูปธรรม

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นแนวทางและประโยชน์สำหรับผู้ที่กำลังศึกษาเกี่ยวกับการออกแบบสถาปัตยกรรมคอมพิวเตอร์และระบบสมองกลฝังตัว ขอขอบพระคุณอาจารย์ศุภดา เทพ สวัสดิพิศาล ที่ได้กรุณาให้ความรู้ คำปรึกษา และการชี้แนะข้อบกพร่องต่าง ๆ ในการทดลองนี้จนเสร็จสมบูรณ์

คณะผู้จัดทำ

สารบัญ

1	บทนำ	4
2	วัตถุประสงค์	4
3	เครื่องมือและอุปกรณ์ที่ใช้	5
3.1	โปรแกรมจำลองการทำงาน	5
3.2	รายการอุปกรณ์ทั้งหมดในวงจร	5
4	แนวคิดการออกแบบวงจร	5
4.1	ขั้นตอนรอบการประมวลผล (3-Stage Cycle)	5
4.2	แผนภาพเส้นทางข้อมูล (Block Diagram)	5
4.3	รูปแบบของ Bit ชุดคำสั่ง (Instruction Format)	7
5	การทำงานของแต่ละส่วนย่อย	7
5.1	รายละเอียด Block ข้อมูลย่อยใน ec1_datapath	7
5.2	โครงสร้าง FSM และ Logic Gate ใน ec1_control	8
6	โปรแกรมใน ROM และผังการไหล	9
6.1	ตารางถอดรหัส ROM Program	9
6.2	ผังการไหลของโปรแกรม (Flowchart)	10
7	ตารางการทำงานที่ละคาบสัญญาณนาฬิกา	11
8	วิธีทดลองใน Logisim-Evolution	12
9	ข้อสังเกตและจุดที่ควรระวัง	13
9.1	ความผิดพลาดทั่วไปของการต่อสายสัญญาณและการถอดสมการ Boolean (Hardware Bugs)	13
9.2	การวิเคราะห์ความพร้อมและความปลอดภัยในการทำงานของหน่วยความจำ	13
9.3	การดัดแปลงและต่อขยายชุดคำสั่ง INC A (Extended Instruction Design)	14
10	สรุปผล	14
11	เอกสารอ้างอิง	14

สารบัญตาราง

ตารางที่ 1	รายการอุปกรณ์ทั้งหมดในวงจรและหน้าที่	5
ตารางที่ 2	ตารางการถอดรหัส ROM Program	9
ตารางที่ 3	การทำงานที่ละคาบสัญญาณนาฬิกา (คาบที่ 1-17)	11
ตารางที่ 4	การทำงานที่ละคาบสัญญาณนาฬิกา (คาบที่ 18-34)	12

สารบัญรูปภาพ

รูปที่ 1	BlockDiagram แสดงเส้นทางข้อมูล (Datapath) ของ Microprocessor EC-1	5
รูปที่ 2	รูปแบบ Bit ของ Instruction Format ของ Microprocessor EC-1	7
รูปที่ 3	วงจรอุปกรณ์และเส้นทางประมวลผล (ec1_datapath) ใน Logisim-Evolution	8
รูปที่ 4	แผนภาพแสดงสถานะและเงื่อนไขการทำงาน (FSM State Diagram)	8
รูปที่ 5	ผังการไหล (Flowchart) แสดงขั้นตอนการประมวลผลโปรแกรมนับถอยหลัง	10
รูปที่ 6	วงจรภาพรวมระดับสูงสุด (main) ของ Microprocessor EC-1	13

ในการออกแบบ Microprocessor สถาปัตยกรรมของหน่วยควบคุม (Control Unit: CU) มีความสำคัญสูงสุดในการบริหารจัดการและจัดลำดับสัญญาณในการประมวลผลคำสั่งต่าง ๆ โดยโครงสร้างการออกแบบดั้งเดิมสามารถจำแนกออกได้เป็นสองวิธีหลักคือ **Hardwired Control Unit** และ **Microprogrammed Control Unit** ซึ่งทั้งสองระบบนี้ได้รับการออกแบบเพื่อแก้ปัญหาระบบคำสั่งที่มีความแตกต่างกันอย่างมีนัยสำคัญ

ระบบ Hardwired Control Unit จะอาศัยวงจร Logic เชิงผสม (Combinational Logic Gate) และหน่วยความจำสถานะ (Flip-Flop) ในการออกแบบเป็น Finite State Machine (FSM) เพื่อขับสัญญาณควบคุมออกมาโดยตรง จุดเด่นของระบบนี้คือความเร็วในการประมวลผลสูงสุดและการสิ้นเปลืองพื้นที่ Hardware ต่ำมาก เนื่องจากไม่มีความจำเป็นต้องดึงรหัสสัญญาณจากหน่วยความจำย่อย แต่มีข้อจำกัดที่ใหญ่ที่สุดคือไม่มีความยืดหยุ่น หากมีคำสั่งใหม่เพิ่มเข้ามาหรือต้องการแก้ไขพฤติกรรม จะต้องทำการออกแบบตัวแปรสัญญาณในรูปสมการ Boolean (Boolean Equations) และจัดวาง Gate ใหม่ทั้งหมดในระดับโครงสร้าง Hardware

ในทางกลับกัน ระบบ Microprogrammed Control Unit (ซึ่งได้ทดลองใน Lab 1) จะแก้ปัญหาคือความยืดหยุ่นนี้โดยการเปลี่ยนสัญญาณควบคุมต่าง ๆ ให้เป็นรหัสข้อมูล (Microprogram) และนำไปเก็บไว้ใน ROM (เรียกว่า Control Store) การทำงานย่อยในแต่ละจังหวะสัญญาณนาฬิกาจะอาศัยการอ่านรหัสสัญญาณคำสั่ง (Microinstruction) จาก Control Store ออกมาใช้งานโดยตรง วิธีนี้มีความยืดหยุ่นสูงมากเนื่องจากสามารถแก้ไขโครงสร้างชุดคำสั่งหรือปรับพฤติกรรมการควบคุม Hardware ได้ง่ายเพียงเปลี่ยนข้อมูลใน ROM โดยไม่ต้องดัดแปลงการต่อสาย Logic Gate แต่ระบบจะช้ากว่าเนื่องจากต้องใช้รอบการอ่านหน่วยความจำ (Memory Access Time) เสมอ

ในการทดลองนี้ จะศึกษาการออกแบบและสร้าง Microprocessor จำลองขนาดเล็กชื่อ **EC-1 (The EC-1 Microprocessor)** ซึ่งเป็นสถาปัตยกรรมแบบย่อส่วน (Minimal CPU Architecture) ขนาดข้อมูล 8 Bit และ Address หน่วยความจำ 4 Bit ที่ขับเคลื่อนสัญญาณด้วยตัวควบคุมประเภท Hardwired FSM ที่พัฒนาด้วย Combinational Logic Gates ร่วมกับ D Flip-Flop

1. เพื่อทำความเข้าใจการเชื่อมต่อและการถ่ายโอนข้อมูลของส่วนประมวลผลข้อมูล (Datapath) ของ Microprocessor จำลองผ่านโปรแกรม Logisim-Evolution
2. เพื่อศึกษาขั้นตอนการวิเคราะห์สถานะ การเขียนผังเชื่อมโยงสถานะ (State Diagram) และการถอดสมการควบคุม (Next-state / Output Equations) ของ Finite State Machine (FSM) สำหรับสถาปัตยกรรมคอมพิวเตอร์ระดับ Hardware
3. เพื่อพัฒนาความรู้ในการทำโปรแกรมระดับต่ำ (Assembly Language) รวมถึงการแปลงให้เป็นรหัสเครื่อง (Machine Code) เพื่อจัดเก็บลงในหน่วยความจำ ROM และจำลองสัญญาณการทำงานรอบนาฬิกาทีละขั้นตอน (Cycle-by-Cycle Execution)

3.1 โปรแกรมจำลองการทำงาน

- **Logisim-Evolution v4.1.0** — โปรแกรมระบบจำลองการทำงานทางดิจิทัลและ Logic ใช้ในการออกแบบ จำลองสถานะ และตรวจสอบพฤติกรรมการแลกเปลี่ยนข้อมูลภายในโครงสร้าง Microprocessor ไฟล์หลักคือ `Lab2_ec1_microprocessor.circ`

- เครื่องมือ Poke Tool — ใช้ในการกระตุ้นสัญญาณนาฬิกา (CLK) และขา Reset (RESET) หรือป้อนค่า Input แบบเรียลไทม์เพื่อสังเกตสัญญาณควบคุม
- Function Auto-Tick — ใช้ในการรันจังหวะสัญญาณนาฬิกาโดยอัตโนมัติด้วยความถี่คงที่เพื่อจำลองการรันโปรแกรมจนสิ้นสุด

3.2 รายการอุปกรณ์ทั้งหมดในวงจร

ชิ้นส่วน Hardware และ Module ทั้งหมดที่ใช้สร้าง Microprocessor EC-1 ได้วิเคราะห์จากไฟลวงจรหลัก สรุปได้ดังตารางที่ 1

ตารางที่ 1 รายการอุปกรณ์ทั้งหมดในวงจรและหน้าที่

อุปกรณ์	Module / พิกัด	การตั้งค่า	หน้าที่การทำงาน
Register PC	ec1_datapath (190,260)	4 Bit	หน่วยนับคำสั่งชี้ Address หน่วยความจำ ROM
Register IR	ec1_datapath (370,180)	8 Bit	ถือข้อมูลคำสั่งจาก ROM เพื่อจ่ายสัญญาณถอดรหัส
Register A	ec1_datapath (680,180)	8 Bit	Accumulator พักและสะสมผลลัพธ์การประมวลผล
Program ROM	ec1_datapath (270,180)	16 × 8 Bit	หน่วยความจำโปรแกรมเก็บรหัสเครื่อง 8 Bit
Subtractor	ec1_datapath (830,190)	8 Bit	ตัวลบค่าสะสมด้วยค่าคงที่ 1 ($A \leftarrow A - 1$)
Zero Comparator	ec1_datapath (760,250)	8 Bit	วงจรตรวจสอบเงื่อนไขค่าศูนย์ ($A \neq 0$)
Tri-state Buffer	ec1_datapath (690,120)	8 Bit	Buffer คุมช่อง Output ภายนอก OUTPUT_A
Multiplexer (IN_mux)	ec1_datapath (600,180)	2 → 1, 8 Bit	เลือกค่าสะสมระหว่าง INPUT_A หรือ A - 1
Multiplexer (JNZ_mux)	ec1_datapath (120,280)	2 → 1, 4 Bit	เลือก Address PC ถัดไประหว่าง PC + 1 หรือ Target
D Flip-Flops (Q2, Q1, Q0)	ec1_control	1 Bit (x3)	หน่วยเก็บสถานะปัจจุบันของหน่วยควบคุม FSM
Logic Gates	ec1_control	AND, OR, NOT	Combinational Logic ถอดสมการ FSM

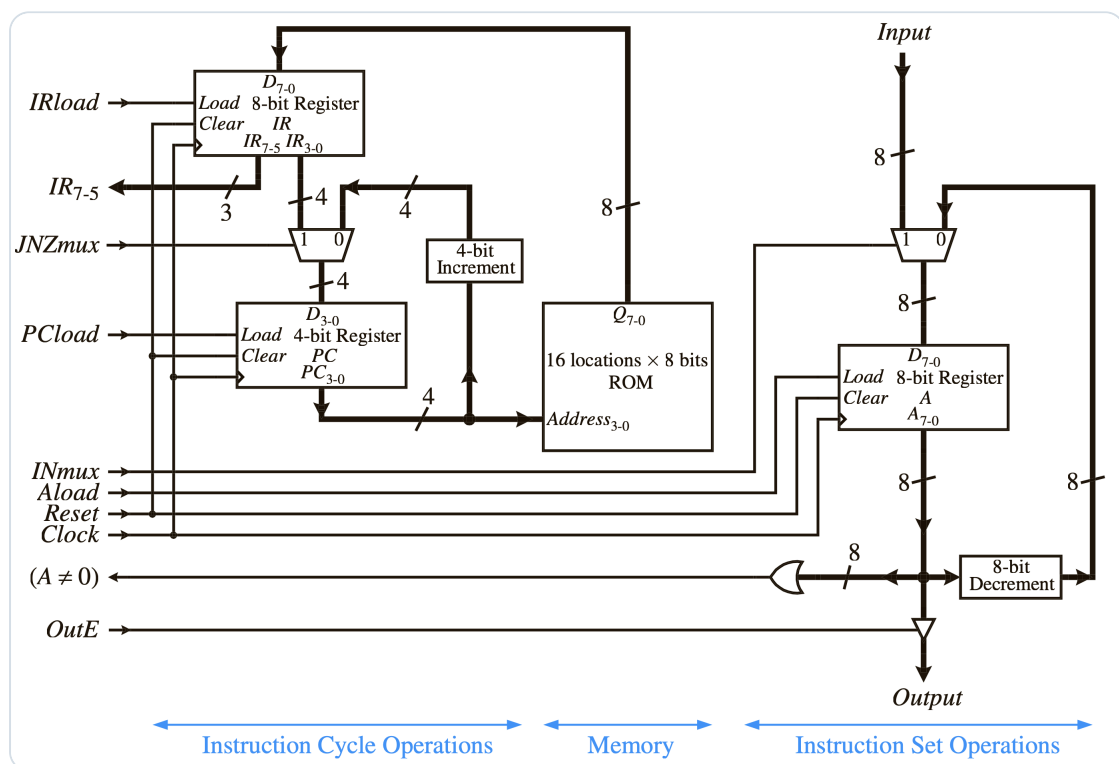
4.1 ขั้นตอนรอบการประมวลผล (3-Stage Cycle)

การประมวลผลสัญญาณคำสั่งและรอบควบคุมการถ่ายโอนข้อมูลของซีพียู EC-1 จะแบ่งออกเป็นรอบการทำคำสั่งย่อย 3 ขั้นตอนดังนี้

1. **FETCH (ขั้นตอนการดึงคำสั่ง):** หน่วย PC จะชี้ไปที่ Address เป้าหมายใน ROM จากนั้นอ่านข้อมูลคำสั่งขนาด 8 Bit มาจัดเก็บที่ Register IR พร้อมกับคำนวณบวกเพิ่มค่าใน PC ถัดไปเตรียมรอไว้ ($PC \leftarrow PC + 1$)
2. **DECODE (ขั้นตอนการถอดรหัสคำสั่ง):** สัญญาณ Opcode 3 Bit จาก IR ('IR[7:5]') จะถูกส่งให้กับ FSM ในหน่วยควบคุม เพื่อนำไปพิจารณาสถานะปัจจุบันและกระตุ้นสายควบคุมในรอบถัดไป
3. **EXECUTE (ขั้นตอนการดำเนินการคำสั่ง):** ชับสัญญาณการโหลดข้อมูล สลับค่าตัวเลือก Input หรือควบคุมกระโดด Address ตามที่ FSM ประมวลผล จากนั้นจะวนกลับส่งสถานะเริ่มวงรอบ FETCH คำสั่งถัดไป

4.2 แผนภาพเส้นทางข้อมูล (Block Diagram)

โครงสร้างสถาปัตยกรรมภายนอกและการถ่ายโอนข้อมูลระหว่างส่วนของ Datapath และ Control Unit แสดงดังรูปที่ 1



รูปที่ 1 — BlockDiagram แสดงเส้นทางข้อมูล (Datapath) ของ Microprocessor EC-1

4.3 รูปแบบของ Bit ชุดคำสั่ง (Instruction Format)

คำสั่งขนาด 8 Bit ของ EC-1 ถูกจัดสรร Bandwidth ออกเป็น 3 ส่วนหลักเพื่อจำหน่ายที่สัญญาณตามรูปที่ 2



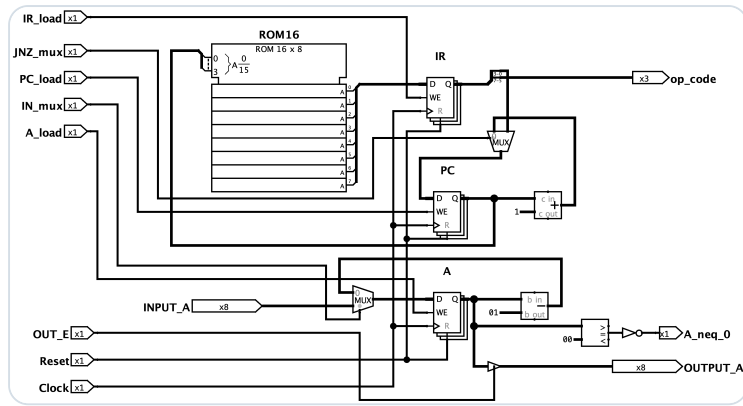
รูปที่ 2 — รูปแบบ Bit ของ Instruction Format ของ Microprocessor EC-1

5 การทำงานของแต่ละส่วนย่อย (Detailed Operation)

5.1 รายละเอียด Block ข้อมูลย่อยใน ec1_datapath

Module ทางเดินข้อมูล 'ec1_datapath' มีส่วนประมวลผลและการโอนถ่ายข้อมูลย่อยที่มีกลไกการควบคุมร่วมกับสัญญาณสถานะดังนี้

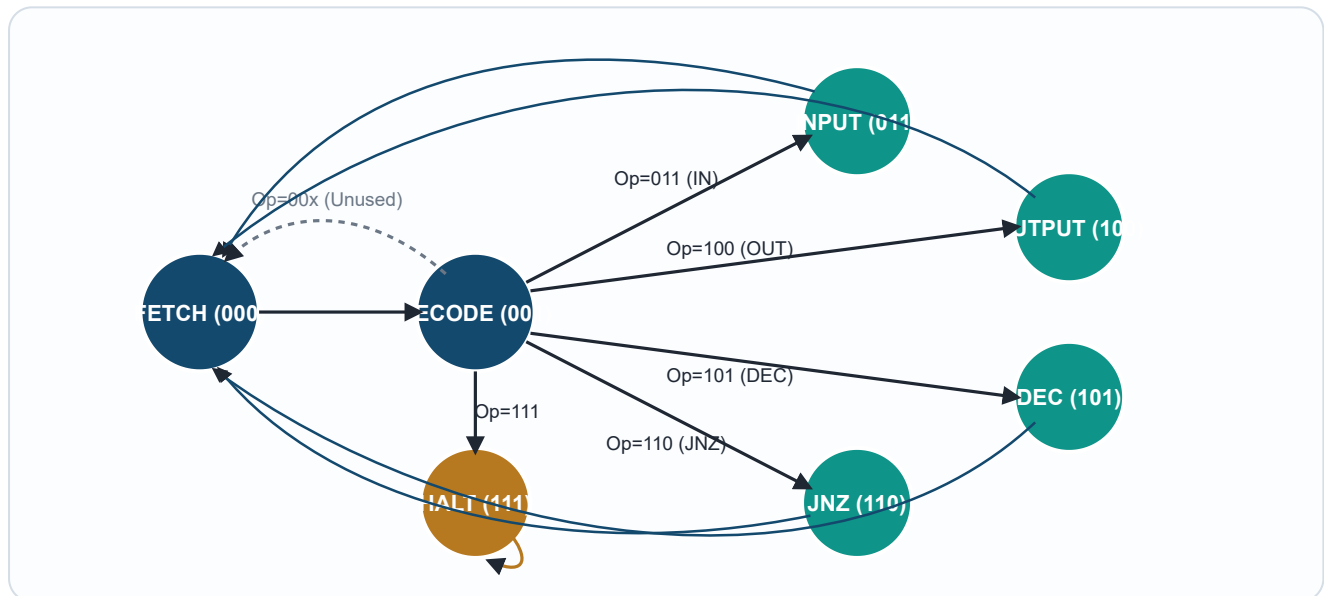
- **Program Counter (PC)** — Register ขนาด 4 Bit ทำหน้าที่เก็บ Address ใน ROM สำหรับดึงคำสั่งปัจจุบัน เมื่อขอบสัญญาณนาฬิกาขยับขึ้น หากสาย 'PC_load = 1' ตัวชี้จะ Update ค่าจากช่อง MUX โดยเลือกช่อง 0 ('PC + 1') หรือช่อง 1 ('Target' Address กระโดด) ขึ้นอยู่กับการประมวลของ 'JNZ_mux'
- **Instruction Register (IR)** — Register ขนาด 8 Bit รับสัญญาณข้อมูลโปรแกรมดิบที่คายออกมาจาก Program ROM และล็อกสัญญาณไว้อุปกรณ์เมื่อควบคุมได้รับ Logic 'IR_load = 1' เพื่อส่งออก Opcode 3 Bit ไประดับถัดไป
- **Program ROM** — Address หน่วยความจำขนาด 16 × 8 Bit ทำหน้าที่บรรจุรหัสเครื่องสำหรับการทดลอง การดึง Address จะตอบสนองทันทีตามค่า PC โดยไม่ต้องอาศัยคาบ Clock ช่วยหน่วย
- **Accumulator A (Register A)** — Register ขนาด 8 Bit สำหรับทำหน้าที่พักข้อมูลการประมวลผลหลัก โหลดค่าเมื่อ 'A_load = 1' ข้อมูลภายในจะเลือกมาจาก Multiplexer 'IN_mux'
- **Subtractor & Zero Comparator** — ชุด Hardware ลบข้อมูลแบบอัตโนมัติ ทำการลบค่าจากสะสมลงหนึ่งค่า ('A - 1') ตลอดเวลาเพื่อรอป้อนกลับ และวงจรเปรียบเทียบขนาด 8 Bit จะสร้างระดับ Logic สถานะเดียว 'A_neq_0' เพื่อส่งให้กับ FSM คอยวิเคราะห์การข้ามสถานะ
- **Tri-state Output Buffer** — อุปกรณ์จำกัดและขับเคลื่อนไฟฟ้า สัญญาณข้อมูลของ Accumulator A จะส่งออกไปยังภายนอก 'OUTPUT_A' ก็ต่อเมื่อได้สัญญาณเปิดสถานะ 'OUT_E = 1' เท่านั้น



รูปที่ 3 — วงจรอุปกรณ์และเส้นทางประมวลผล (ec1_datapath) ใน Logisim-Evolution

5.2 โครงสร้าง FSM และ Logic Gate ใน ec1_control

ส่วนควบคุม 'ec1_control' ใช้ D Flip-Flops 3-bit ('Q2 Q1 Q0') เก็บรักษาและระบุค่าของวงจรรอบการทำงาน โดยโครงสร้างการแมป Opcode เข้ากับหมายเลขสถานะช่วยอำนวยความสะดวกในการจัดรูปแบบ Combinational Logic ของวงจร Logic ควบคุม สมผังวิเคราะห์ FSM ในรูปที่ 4



รูปที่ 4 — แผนภาพแสดงสถานะและเงื่อนไขการทำงาน (FSM State Diagram) ของหน่วยควบคุม EC-1

สมการ Logic สำหรับการข้ามเปลี่ยนสถานะ (Next-state Equations) ที่ถอดได้จาก FSM มีรูปแบบความสัมพันธ์ดังนี้:

$$\begin{aligned} D2 &= \text{Decode} \cdot 02 + \text{Halt} \\ D1 &= \text{Decode} \cdot 01 + \text{Halt} \\ D0 &= \text{Fetch} + \text{Decode} \cdot 00 \cdot (02 + 01) + \text{Halt} \end{aligned}$$

สัญญาณควบคุมภายนอกสำหรับการส่งให้ระบบ Datapath มี FunctionLogic ความสัมพันธ์กับรหัสสถานะดังต่อไปนี้:


```

IR_load = Fetch
PC_load = Fetch + JNZ · A_neq_0
IN_mux = Input
A_load = Input + Dec
JNZ_mux = JNZ
Out_E = Output
Halt = Halt

```

6 โปรแกรมใน ROM และผังการไหล (ROM Program & Flowchart)

6.1 ตารางถอดรหัส ROM Program

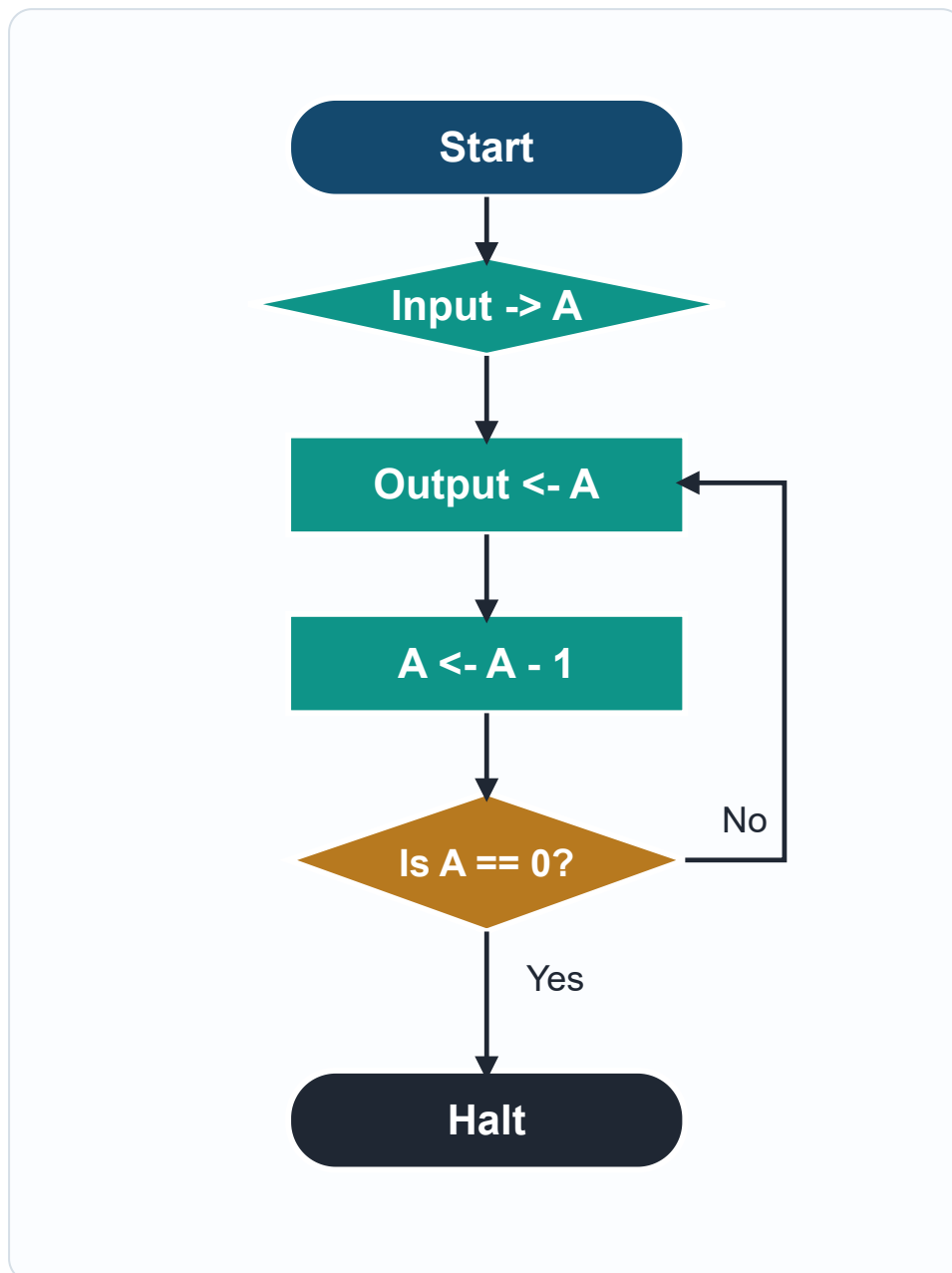
รหัสคำสั่งสำหรับการทดลองการนับถอยหลังของ Accumulator A มี Parameter การแมปไบนารีและรหัส Hex ดังตารางที่ 2

ตารางที่ 2 ตารางการถอดรหัส ROM Program

ROM Addr	Assembly Mnemonic	Machine Code (Hex)	Opcode (Bin)	Operand (Bin)	ความหมายการทำงาน
0x0	IN A	0x60	011	0000	รอข้อมูลจาก Input Port ภายนอก แล้วนำมาตั้งค่าให้สะสมใน A
0x1	OUT A	0x80	100	0000	แสดงค่าข้อมูลจาก Accumulator A ออกที่ Output Port ภายนอก
0x2	DEC A	0xA0	101	0000	ลดค่าใน Accumulator A ลง 1 ค่า ($A \leftarrow A - 1$)
0x3	JNZ 0x1	0xC1	110	0001	หากค่าใน Accumulator $A \neq 0$ ให้เปลี่ยน PC ย้อนกลับไป Address 0x1
0x4	HALT	0xE0	111	0000	สั่งหยุดการยับสถานะของซีพียูเข้าสู่สถานะหยุดทำงานถาวร

6.2 ผังการไหลของโปรแกรม (Flowchart)

ผังการประมวลผลคำสั่งเชิงโปรแกรม (Software Execution Flow) เพื่อการลดค่า countdown แสดงไว้ในรูปที่ 5



รูปที่ 5 — ผังการไหล (Flowchart) แสดงขั้นตอนการประมวลผลโปรแกรมนับถอยหลัง

การจำลองการเปลี่ยนแปลงทางข้อมูลและสัญญาณในระบบในรูปสัญญาณนาฬิกา (Cycle-by-Cycle Trace) โดยเริ่มจากการป้อนค่าคงที่ Input เป้าหมายล่วงหน้าคือ 'INPUT_A = 3' สามารถระบุ Parameter สัญญาณขาออก ณ ขอบสัญญาณขาขึ้นของแต่ละจังหวะเวลาได้ดังนี้

ตารางที่ 3 การทำงานที่ละคาบสัญญาณนาฬิกา (คาบที่ 1-17)

Cycle	FSM State	Q210	PC	IR	A	Output	JNZmx	INmx	Aload	PCload	IRload	OutE
1	FETCH	000	0	00	0	Hi-Z	0	0	0	1	1	0
2	DECODE	001	1	60	0	Hi-Z	0	0	0	0	0	0
3	INPUT	011	1	60	0	Hi-Z	0	1	1	0	0	0
4	FETCH	000	1	60	3	Hi-Z	0	0	0	1	1	0
5	DECODE	001	2	80	3	Hi-Z	0	0	0	0	0	0
6	OUTPUT	100	2	80	3	3	0	0	0	0	0	1
7	FETCH	000	2	80	3	Hi-Z	0	0	0	1	1	0
8	DECODE	001	3	A0	3	Hi-Z	0	0	0	0	0	0
9	DEC	101	3	A0	3	Hi-Z	0	0	1	0	0	0
10	FETCH	000	3	A0	2	Hi-Z	0	0	0	1	1	0
11	DECODE	001	4	C1	2	Hi-Z	0	0	0	0	0	0
12	JNZ	110	4	C1	2	Hi-Z	1	0	0	1	0	0
13	FETCH	000	1	C1	2	Hi-Z	0	0	0	1	1	0
14	DECODE	001	2	80	2	Hi-Z	0	0	0	0	0	0
15	OUTPUT	100	2	80	2	2	0	0	0	0	0	1
16	FETCH	000	2	80	2	Hi-Z	0	0	0	1	1	0
17	DECODE	001	3	A0	2	Hi-Z	0	0	0	0	0	0

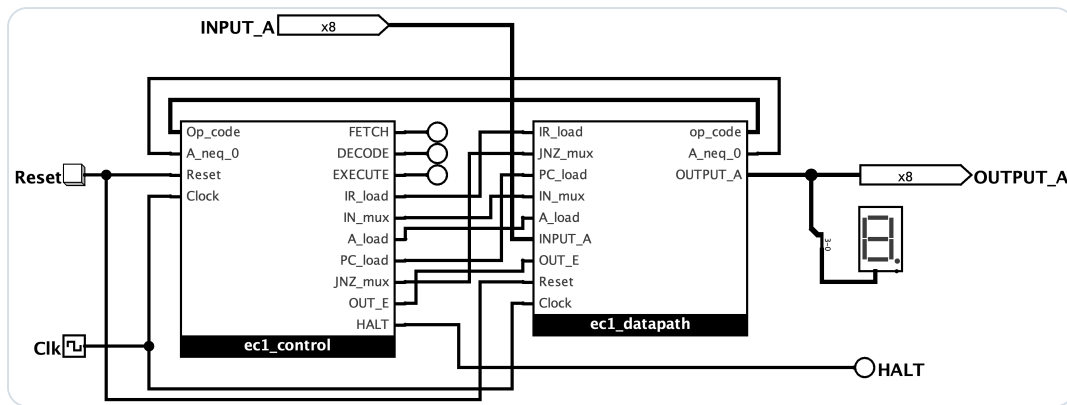
ตารางที่ 4 การทำงานที่ละคาบสัญญาณนาฬิกา (คาบที่ 18-34)

Cycle	FSM State	Q210	PC	IR	A	Output	JNZmx	INmx	Aload	PCload	IRload	OutE
18	DEC	101	3	A0	2	Hi-Z	0	0	1	0	0	0
19	FETCH	000	3	A0	1	Hi-Z	0	0	0	1	1	0
20	DECODE	001	4	C1	1	Hi-Z	0	0	0	0	0	0
21	JNZ	110	4	C1	1	Hi-Z	1	0	0	1	0	0
22	FETCH	000	1	C1	1	Hi-Z	0	0	0	1	1	0
23	DECODE	001	2	80	1	Hi-Z	0	0	0	0	0	0
24	OUTPUT	100	2	80	1	1	0	0	0	0	0	1
25	FETCH	000	2	80	1	Hi-Z	0	0	0	1	1	0
26	DECODE	001	3	A0	1	Hi-Z	0	0	0	0	0	0
27	DEC	101	3	A0	1	Hi-Z	0	0	1	0	0	0
28	FETCH	000	3	A0	0	Hi-Z	0	0	0	1	1	0
29	DECODE	001	4	C1	0	Hi-Z	0	0	0	0	0	0
30	JNZ	110	4	C1	0	Hi-Z	1	0	0	0	0	0
31	FETCH	000	4	C1	0	Hi-Z	0	0	0	1	1	0
32	DECODE	001	5	E0	0	Hi-Z	0	0	0	0	0	0
33	HALT	111	5	E0	0	Hi-Z	0	0	0	0	0	0
34	HALT	111	5	E0	0	Hi-Z	0	0	0	0	0	0

8 วิธีทดลองใน Logisim-Evolution (Simulation Procedure)

การจำลองการทำงานระดับปฏิบัติการของโครงสร้าง Hardware ซีพียู EC-1 มีกระบวนการดำเนินงานหลักดังนี้:

1. **การเตรียมโครงสร้างวงจร (Main Design Sheet):** ทำการลาก Module 'ec1_datapath' และ 'ec1_control' มาเชื่อมต่อร่วมกับปุ่ม RESET และ สัญญาณนาฬิกาหลักของ Sheet 'main'
2. **การแก้ไขรหัสโปรแกรมใน ROM:** คลิกขวาที่ ROM ของ Datapath และพิมพ์รหัส Hex ของโปรแกรม: **60 80 a0 c1 e0** แล้วเคลียร์ค่า Address ที่เหลือ เป็น 00 ทั้งหมด
3. **การเริ่มทำงานและเตรียมข้อมูล Input:** ใช้เครื่องมือ Poke Tool ตั้งค่าเริ่มต้นที่ Port 'INPUT_A' (ตัวอย่าง: ตั้งค่าเป็น 3) จากนั้นกดขวา Reset Reset ให้ เป็น 1 แล้วกลับเป็น 0 เพื่อเริ่มวงรอบ PC จากตำแหน่งแรก
4. **การขยับจังหวะสัญญาณนาฬิกา:** สลับระดับสัญญาณ Clock 0 ↔ 1 ต่อเนื่อง หรือผ่านเมนู **Simulate → Auto-Tick Enabled** สังเกต Output และ เช็คสถานะหยุดการทำงานที่ค่า HALT



รูปที่ 6 — วงจรภาพรวมระดับสูงสุด (main) ของ Microprocessor EC-1 พร้อมจอแสดงผล Hex

9

ข้อสังเกตและจุดที่ควรระวัง (Observations and Caveats)

9.1 ความผิดพลาดทั่วไปของการต่อสายสัญญาณและการถอดสมการ Boolean (Hardware Bugs)

- การสูญเสียการตัดสินใจกระโดดมีเงื่อนไข JNZ ในการคุม PC: จุดที่ควรระวังสูงสุดคือหากนักศึกษาปรับลดทอนสัญญาณโดยนำสมการ 'PC_load' ควบเข้ากับเงื่อนไข FETCH เท่านั้น สัญญาณตัดสินใจกระโดดแบบมีเงื่อนไขของคำสั่ง 'JNZ' จะไม่ส่งผลกระโดด ทำให้โปรแกรม countdown ไม่ย้อน Loop ขึ้นไปประมวลผล Output ซ้ำ ผลลัพธ์บน Hex Display จะแสดงเลขตัวแรกเพียงครั้งเดียวแล้วจบทำงานทันที
- การประชิด Bit ในช่วงขอบสัญญาณนาฬิกา (Race Conditions): เนื่องจากตำแหน่ง PC และ Register IR รับสัญญาณโหนดที่ขอบขาขึ้นร่วมกัน ทว่าการจำลองวงจรในระดับ LogicSoftware อุปกรณ์ ROM จะยังคงยศาสตร์คำสั่งเดิมของ Address เก่าก่อนการเพิ่มขึ้นของ PC ได้อย่างถูกต้องและทันทั่วทั้ง

9.2 การวิเคราะห์ความพร้อมและความปลอดภัยในการทำงานของหน่วยความจำ (Instruction Wrap-Around)

ในโปรแกรม countdown Loop ประมวลผลถูกออกแบบให้ตรวจสอบเช็คด้วยตัวแปรเปรียบเทียบ 'A != 0' เมื่อ Accumulator A ลดค่าลงถึงศูนย์ คำสั่ง 'JNZ' จะไม่ส่งผลกระโดด แต่หาก FSM ถูกป้อนค่าและสั่งควบคุมลบข้อมูลย้อนกลับต่อเนื่องไปเรื่อย ๆ โดยไม่มีการจำกัดรอบ ค่าของ Accumulator A จะเกิดเหตุการณ์ **Wrap-around (หรือ Underflow)** จาก '0x00' ข้ามไปเป็นค่าล้นบนสุด '0xFF' (หรือ 255) ทันที ซึ่งอาจก่อให้เกิดจุดบกพร่องร้ายแรงในโปรแกรมระดับบนได้

9.3 การดัดแปลงและต่อขยายชุดคำสั่ง INC A (Extended Instruction Design)

หน่วยควบคุม FSM ของ EC-1 มี Slot ว่างของ Opcode ที่สามารถขยายเพื่อรองรับคำสั่งเพิ่มขึ้นได้ ในที่นี้เสนอการเพิ่มคำสั่งใหม่ **INC A (Increment Accumulator A: $A \leftarrow A + 1$)** โดยการจัดสรรและถอดรูป Boolean ควบคุมมีโครงสร้างดังนี้:

สมการ Logic ระดับ FSM ที่ผ่านการขยายเพื่อรองรับ INC A:

$$D2 = \text{Decode} \cdot 02 + \text{Halt}$$

$$D1 = \text{Decode} \cdot 01 + \text{Halt}$$

$$D0 = \text{Fetch} + \text{Decode} \cdot 00 \cdot (02 + 01) + \text{Halt}$$

สัญญาณควบคุมการเชื่อมต่อทางเดินข้อมูลแบบใหม่:

เพิ่มชุดตัวเลือก Multiplexer จากเดิม 2:1 เป็น 3:1 เพื่อรวมสายสัญญาณ 'A + 1' เข้ามาด้วย โดยมีสมการควบคุมดังนี้:

$$- IN_mux1 = State_2 \text{ (รหัสสถานะ Step คำสั่ง INC)}$$

$$- IN_mux0 = State_3 \text{ (รหัสสถานะ Step คำสั่ง INPUT)}$$

$$- A_load = State_2 + State_3 + State_5 \text{ (INC + INPUT + DEC)}$$

10 สรุปผล (Conclusion)

การปฏิบัติการทดลองวิเคราะห์และสร้างระบบคำสั่งใน Microprocessor EC-1 ประสบผลสำเร็จด้วยดี ผลการจำลองการทดสอบร่วมกับโปรแกรม countdown เลข 3 ถึง 1 สอดคล้องตามทฤษฎีในตาราง Trace คาบเวลาอย่างสม่ำเสมอ โดยระบบส่งผ่านทิศทางข้อมูลของ Accumulator A ไปลดค่าและย้อน Loop กลับมาพิมพ์สัญญาณได้ตามสัญญาณ 'A_neq_0' และเข้าสู่สถานะ Halt ตลอดช่วงการปัดวงจร ณ คาบเวลาสัญญาณนาฬิกาที่ 33 พอดี

ความแตกต่างระหว่างการทดลองรอบนี้กับ Lab 1 สะท้อนให้เห็นว่า การพัฒนา Microprocessor จำลองที่มี FSM ร่วมกับชุดคำสั่ง Assembly ช่วยเอื้อความสะดวกสำหรับนักพัฒนาด้าน Software ในการปรับเขียนโค้ดโดยไม่ต้องกังวลกับการจัดการคำสั่งสัญญาณ Logic ระดับควบคุมเชิงไฟฟ้า (Control Signals) โดยตรงแบบเครื่องใน Lab 1 ตัว FSM ทำหน้าที่เป็นมิดเดิลแวร์ซ่อนความซับซ้อนของทางสัญญาณผ่านรหัสถอดรหัส Opcode ส่งผลให้การเชื่อมโยงระบบสมองกลฝังตัวมีความเป็นระบบและมีประสิทธิภาพในการประมวลผลสูงขึ้นอย่างเด่นชัด

11 เอกสารอ้างอิง (References)

- คทาเทพ สวัสดิพิศาล, "คู่มือการปฏิบัติการทดลองที่ 2: The EC-1 Microprocessor," ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าพระนครเหนือ, 2569.
- E. O. Hwang, *Digital Logic and Microprocessor Design with Interfacing*, 2nd ed. Boston, MA: Cengage Learning, 2018.
- ภาควิชาวิศวกรรมคอมพิวเตอร์, "เอกสารประกอบการเรียนวิชา Embedded System Design: สถาปัตยกรรม Microprocessor เบื้องต้น," สถาบันเทคโนโลยีพระจอมเกล้าพระนครเหนือ, 2569.